Universidad Rafael Landívar.

Facultad de Ingeniería.

Licenciatura en Ingeniería en Informática y Sistemas.

Manejo e implementación de archivos

Docente: Ing. Manuel Rojas

Proyecto 2

Estudiante: Fabritzio Alejandro López Castillo

Carné: 1507525

Quetzaltenango, 9 de noviembre de 2026

1. Estructura del registro de tamaño fijo

Para habilitar el acceso aleatorio sobre el archivo data/contactos.dat sin necesidad de recorrerlo de forma secuencial, todos los contactos se almacenan empleando una estructura de longitud fija estrictamente de 120 bytes codificados en formato UTF-8. Cada campo dispone de un ancho predefinido y se rellena hacia la derecha con caracteres de espacio ASCII (0x20) mediante el método ljust hasta completar su longitud asignada.

Campo	Tipo Lógico	Rango de Bytes	Largo Fijo	Regla de Formato y Relleno
id	Entero único	Bytes 0 – 9	10 bytes	Convertido a texto y rellenado con espacios a la derecha con ljust
nombre	Cadena UTF-8	Bytes 10 – 44	35 bytes	Recorte preventivo en bytes UTF-8 y relleno con espacios mediante ljust
telefono	Cadena UTF-8	Bytes 45 – 59	15 bytes	Dígitos y signos telefónicos ajustados a 15 bytes con ljust
correo	Cadena UTF-8	Bytes 60 – 94	35 bytes	Dirección electrónica rellenada con espacios hasta 35 bytes.
ciudad	Cadena UTF-8	Bytes 95 – 119	25 bytes	Nombre de ciudad ajustado exactamente a 25 bytes en UTF-8.
Tamaño Total del Registro Fijo			120 bytes	Sin saltos de línea intermedios, bloques de 120 bytes.

Dado que en la codificación UTF-8 los caracteres con tilde o la letra ñ ocupan 2 bytes en lugar de 1, el método ajustar_campo() de la clase Contacto calcula los espacios faltantes midiendo la longitud del arreglo de bytes codificado (len(texto.encode("utf-8"))) antes de ljust(). Esto garantiza que ningún carácter especial altere el tamaño físico de 120 bytes en disco.

```python
class Contacto:
    LEN_ID = 10
    LEN_NOMBRE = 35
    LEN_TELEFONO = 15
    LEN_CORREO = 35
    LEN_CIUDAD = 25
    TAMANO_REGISTRO = LEN_ID + LEN_NOMBRE + LEN_TELEFONO + LEN_CORREO + LEN_CIUDAD  # 120 bytes

    def __init__(self, id_contacto: int, nombre: str, telefono: str, correo: str, ciudad: str):
        self.id = int(id_contacto)
        self.nombre = nombre.strip()
        self.telefono = telefono.strip()
        self.correo = correo.strip()
        self.ciudad = ciudad.strip()
```

```python
@staticmethod
def _ajustar_campo(valor: str, largo_bytes: int):
    #Si excede el máximo y rellena con espacios usando ljust
    texto = str(valor).strip()
    while len(texto.encode("utf-8")) > largo_bytes:
        texto = texto[:-1]
    espacios_faltantes = largo_bytes - len(texto.encode("utf-8"))
    return texto.ljust(len(texto) + espacios_faltantes)

def a_registro_fijo(self):
    #Convierte el contacto en una cadena de longitud fija exacta
    return (
        self._ajustar_campo(str(self.id), self.LEN_ID)
        + self._ajustar_campo(self.nombre, self.LEN_NOMBRE)
        + self._ajustar_campo(self.telefono, self.LEN_TELEFONO)
        + self._ajustar_campo(self.correo, self.LEN_CORREO)
        + self._ajustar_campo(self.ciudad, self.LEN_CIUDAD)
    )
```

2. Implementación del acceso aleatorio con seek y tell

El módulo persistencia/gestor_archivos.py administra el archivo contactos.dat en modo binario de lectura y escritura combinada ("rb+") para controlar con exactitud milimétrica el puntero de archivo del sistema operativo y evitar conversiones automáticas de fin de línea en Windows.

Fórmula de Direccionamiento Directo:
Desplazamiento en bytes = numero_registro × 120

2.1 Inserción al final y captura de posición con tell()

Al agregar un nuevo contacto, se posiciona el cabezal exactamente al final del archivo con archivo.seek(0, os.SEEK_END) y se invoca archivo.tell() para conocer el byte exacto donde inicia el bloque. Dividiendo dicho byte entre 120 se obtiene el número de registro secuencial que se almacena en el índice primario.

```
self._asegurar_archivo()
try:
    with open(self.ruta_datos, "rb+") as archivo:
        archivo.seek(0, os.SEEK_END)
        posicion_bytes = archivo.tell()

        if posicion_bytes % Contacto.TAMANO_REGISTRO != 0:
            raise RuntimeError("El archivo contactos.dat está corrupto.")

        numero_registro = posicion_bytes // Contacto.TAMANO_REGISTRO
        datos_utf8 = contacto.a_registro_fijo().encode("utf-8")
        archivo.write(datos_utf8)

    self.gestor_indice.registrar_id(contacto.id, numero_registro)
    return numero_registro, posicion_bytes
```

2.2 Lectura directa y modificación en el lugar:

Tanto para buscar un contacto como para modificarlo, la aplicación consulta primero el número de registro en el índice, multiplica ese valor por 120 y salta directamente al byte objetivo mediante archivo.seek(desplazamiento). Enseguida se verifica la ubicación real del cabezal con archivo.tell(). Al editar un contacto únicamente se sobrescriben los 120 bytes correspondientes a ese registro sin tocar ni reescribir el resto del archivo.

```python
class GestorArchivos:
    def buscar_por_id(self, id_contacto: int):
        #Consulta el índice, hace seek() directo
        numero_registro = self.gestor_indice.obtener_numero_registro(id_contacto)
        if numero_registro is None:
            raise KeyError(f"No existe ningún contacto con el ID {id_contacto}.")

        if not os.path.exists(self.ruta_datos):
            raise FileNotFoundError("El archivo contactos.dat no existe.")

        desplazamiento = numero_registro * Contacto.TAMANO_REGISTRO
        try:
            with open(self.ruta_datos, "rb") as archivo:
                archivo.seek(desplazamiento)
                posicion_actual = archivo.tell()
                bloque = archivo.read(Contacto.TAMANO_REGISTRO)

                if len(bloque) < Contacto.TAMANO_REGISTRO:
                    raise RuntimeError("Registro incompleto o archivo contactos.dat truncado.")

                contacto = Contacto.desde_bytes(bloque)
                return contacto, numero_registro, posicion_actual
```

```python
    def modificar_contacto(self, contacto: Contacto):
        #Sobrescribe únicamente el registro en su posición sin reescribir el resto del archivo
        numero_registro = self.gestor_indice.obtener_numero_registro(contacto.id)
        if numero_registro is None:
            raise KeyError(f"No se puede modificar: el ID {contacto.id} no existe en el índice.")

        if not os.path.exists(self.ruta_datos):
            raise FileNotFoundError("El archivo contactos.dat no existe.")

        desplazamiento = numero_registro * Contacto.TAMANO_REGISTRO
        try:
            with open(self.ruta_datos, "rb+") as archivo:
                archivo.seek(desplazamiento)
                posicion_actual = archivo.tell()
                datos_utf8 = contacto.a_registro_fijo().encode("utf-8")
                archivo.write(datos_utf8)
                return numero_registro, posicion_actual
        except PermissionError as e:
            raise PermissionError("No hay permisos de escritura en contactos.dat.") from e
```

3. Funcionamiento del índice primario y análisis de eficiencia

El archivo data/indice.json actúa como una tabla de direccionamiento primario que asocia la clave única de cada contacto (id) con su posición física de bloque (numero_registro) dentro de contactos.dat. Cada vez que se inserta un contacto nuevo, el diccionario JSON se actualiza inmediatamente en disco.

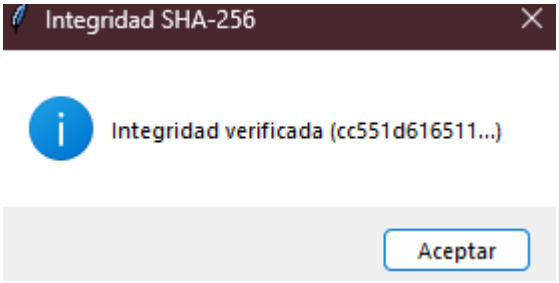
En una búsqueda secuencial tradicional sin índice, localizar un contacto requiere leer y decodificar registro por registro desde el byte 0 hasta encontrar el identificador buscado, lo que implica una complejidad lineal de O(n) operaciones de lectura en disco. En cambio al utilizar un índice primario combinado con seek(), el identificador se resuelve en un diccionario hash con costo promedio O(1) y se realiza un único salto de cabezal en disco directamente al byte exacto, logrando complejidad constante O(1) sin importar si el archivo almacena 10 o 100,000 contactos.

Criterio de Comparación	**Búsqueda Secuencial en Archivo**	**Búsqueda con Índice Primario + seek**
Complejidad temporal	Lineal O(n) respecto al total de contactos.	Constante O(1) en resolución de clave y salto.
Lecturas en disco (peor caso)	n lecturas de 120 bytes (recorre todo el archivo).	1 sola lectura exacta de 120 bytes tras seek.
Bytes transferidos (ej. 10,000 registros)	Hasta 1,200,000 bytes leídos del disco.	Exactamente 120 bytes leídos de contactos.dat.
Escalabilidad	El tiempo de respuesta se degrada proporcionalmente al tamaño.	Tiempo de respuesta inmediato e invariable.

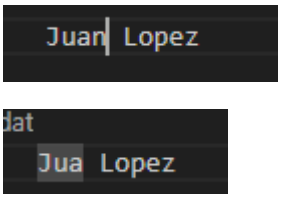
4. Verificación de integridad con SHA-256

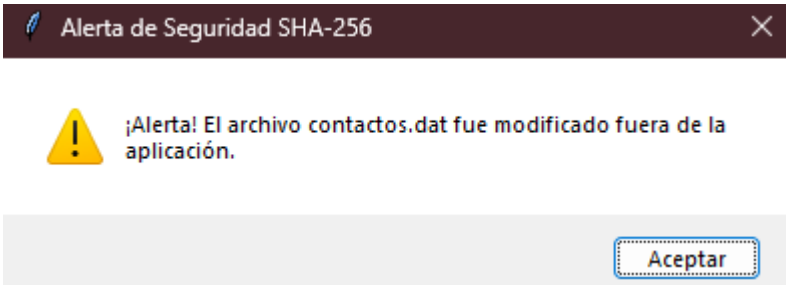
Para proteger los datos frente a alteraciones externas, el módulo servicios/servicio_integridad.py utiliza hashlib.sha256() para calcular el resumen criptográfico de contactos.dat cada vez que se agrega o modifica un registro, almacenando la huella resultante en data/contactos.sha256. Al iniciar la aplicación (o desde el menú Herramientas), el sistema recalcula el hash actual del archivo y lo compara contra el valor resguardado.

Operación normal:



Edición de contactos.dat en un editor externo:





5. Comparación del archivo original con el respaldo ZIP

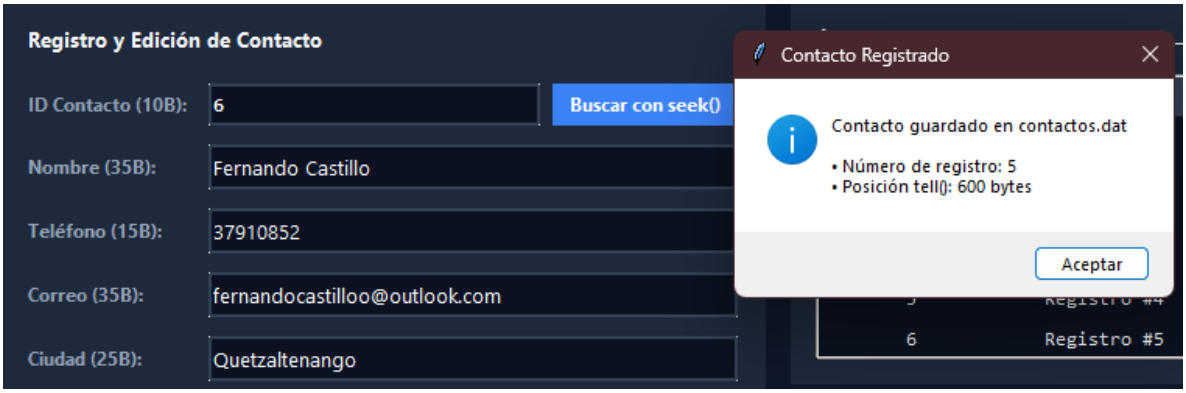
La herramienta de respaldo empaqueta tanto data/contactos.dat como data/indice.json en un archivo comprimido dentro de la carpeta exportados/. Debido a que los registros de tamaño fijo contienen secuencias repetitivas de espacios en blanco (0x20) generadas por ljust, el algoritmo de compresión logra una alta tasa de reducción de espacio.

Archivo	**Tamaño en Disco (Bytes)**	**Observaciones Técnicas**
data/contactos.dat	600 bytes	N registros × 120 bytes fijos por contacto.
data/indice.json	68 bytes	Diccionario JSON con mapeo de ID a número de registro.
Tamaño Original Total	668 bytes	Suma obtenida
Archivo respaldo_YYYYMMDD.zip	505 bytes	Comprimido.
Porcentaje de Reducción / Ahorro	24.4 %	Alta compresión de los espacios de relleno de tamaño fijo.

6. Capturas de pantalla del funcionamiento del sistema

A continuación se documenta la ejecución de las operaciones principales: inserción y búsqueda directa mostrando la posición de tell, modificación en el lugar, exportación a JSON y reporte PDF, y consulta de metadatos y permisos del archivo.

Registro de un nuevo contacto:



Búsqueda de un contacto:



Modificacion de un contancto:



Archivo JSON generado:



exportados > {} contactos_exportados.json > ...

```
[
    {
        "id": 1,
        "nombre": "Juan Lopez",
        "telefono": "12345678",
        "correo": "jjuanl@gmail.com",
        "ciudad": "Quetzaltenango"
    },
    {
        "id": 2,
        "nombre": "Maria Sanchez",
        "telefono": "78965412",
        "correo": "sanchezmaria@outlook.com",
        "ciudad": "San Marcos"
    },
    {
        "id": 3,
        "nombre": "José Barrios",
        "telefono": "12547850",
        "correo": "josbarr@gmail.com",
        "ciudad": "Ciudad de Guatemala"
    },
    {
        "id": 4,
```

Archivo PDF generado:



Reporte de Agenda de Contactos

Fecha de generación: 08/11/2026 13:38:09

ID	Nombre	Teléfono	Correo	Ciudad
1	Juan Lopez	12345678	jjuanl@gmail.com	Quetzaltenango
2	Maria Sanchez	78965412	sanchezmaria@outlook.com	San Marcos
3	José Barrios	12547850	josbarr@gmail.com	Ciudad de Guatem
4	Pedro Ordoñez	89654035	pedroooordoñezz@outlook.com	Quetzaltenango
5	Sofía Morales	00452399	sofimorales@gmail.com	Antigua Guatemal
6	Fernando Castillo	37910852	fernandocastilloo@outlook.co	Quetzaltenango

Ventana de información del archivo y permisos del sistema:

